Lecture Notes: Reinforcement Learning

Course: 2110573 Pattern Recognition

2026-04-02
version 0.1

1 Introduction to Reinforcement Learning

1.1 What is Reinforcement Learning (RL)?

Reinforcement Learning (RL) can be understood from three different perspectives:

1. **A Problem:** Formulating how an agent ought to take actions in an environment to maximize cumulative reward.
2. **A Community:** A dedicated group of researchers and practitioners working on solving this problem.
3. **A Set of Methods:** Algorithms produced by the community which can often be applied to a wide array of other, general optimization problems.

1.2 The Three Modes of Learning

Machine learning is traditionally divided into three distinct modes:

- **Supervised Learning:** The algorithm is trained on a labeled dataset. It observes pairs $(x_1, y_1), (x_2, y_2), \dots$ and its objective is to predict y_{new} for an unseen input x_{new}.
- **Unsupervised Learning:** The algorithm observes unannotated data $x_1, x_2, x_3, \dots$ and its objective is to model the underlying distribution $P(x)$, find a good representation of the data (e.g., clustering), or learn underlying structures.
- **Reinforcement Learning:** The algorithm observes states $(s_1, s_2, s_3, \dots)$ and rewards $(r_1, r_2, r_3, \dots)$ while taking actions $(a_1, a_2, a_3, \dots)$. The objective is to find the best sequence of actions to yield the highest cumulative reward.

1.3 RL vs. Other Modes of Learning

RL differs from supervised and unsupervised learning in several crucial ways:

- It involves **sequential decisions**, where current actions affect future states.
- You are given a **goal** (maximize reward) rather than explicit instructions (labels) on how to achieve it.
- There is no traditional concept of a "training set" and "test set".
- It requires **Active Learning** (the agent interacts with the environment) rather than Passive Learning (static datasets).

Yann LeCun's Cake Analogy: To illustrate the amount of information given to the machine during learning, Yann LeCun compares AI to a cake:

- **Self-Supervised Learning (The Cake):** Predicts any part of the input from any observed part. Millions of bits per sample.
- **Supervised Learning (The Icing):** Predicts human-supplied labels. 10 to 10,000 bits per sample.
- **Reinforcement Learning (The Cherry):** Predicts a scalar reward given only occasionally. A few bits for some samples.

1.4 Applications of RL

RL has been successfully deployed in various domains:

- **Games:** AlphaGo, Chess, StarCraft, Dota 2, Poker.
- **Finance:** Algorithmic trading (e.g., JP Morgan's LOXM).
- **Robotics:** Learning complex manipulation and locomotion.
- **Computer Systems:** Data center and resource management, system configuration, DRAM controllers.
- **Online Services:** Ad bidding, recommender systems (e.g., Multi-Armed Bandits).
- **Sciences & NLP:** Optimizing chemical reactions, guiding large language models (like ChatGPT via RLHF), and AutoML.

2 The Reinforcement Learning Framework

2.1 Agent-Environment Interaction

The core of RL is learning through trial and error. The framework consists of an **Agent** interacting with an **Environment**:

- The agent observes the current **State** (s_t) from the environment.
- The agent takes an **Action** (a_t).
- The environment transitions to a new state and returns a **Reward** (r_t) to the agent.

This process repeats sequentially: $s_0 \xrightarrow{a_0} r_1, s_1 \xrightarrow{a_1} r_2, s_2 \ldots$

2.2 Markov Decision Process (MDP)

The environment is formally modeled as a Markov Decision Process, defined by a 5-tuple (S, A, P, R, γ):

- **S**: Set of all possible states.
- **A**: Set of all possible actions.
- **P**: Transition probabilities. The probability of moving to state s_{t+1} given state s_t and action a_t: $P^a_{s,s'} = \text{Prob}[s_{t+1} = s' \mid s_t = s, a_t = a]$. The environment can be stochastic (one action can lead to multiple outcomes).
- **R**: Reward function associated with states and actions.
- γ: Discount factor ($\gamma \in [0, 1]$), dictating the present value of future rewards.

The Markov Property: A state transitions under the Markov property if $p(s_{t+1} \mid s_t, a_t)$ depends *only* on the immediate prior state s_t and action a_t, completely ignoring the history before t.

- **Fully Observable State:** All information from the past is captured in the current state (e.g., a Go board position, or stacking the last few frames of an Atari game). The Markov property holds.
- **Partially Observable State:** The agent cannot see the full environment (e.g., the "Fog of War" in strategy games). The standard Markov property does not strictly hold without memory mechanisms.

3 Policies and Expected Return

3.1 Policy (π)

A policy is a mapping from a state to an action. It defines the agent's behavior.

- **Deterministic Policy:** $a = \pi(s)$. Always maps a state to the exact same action.
- **Stochastic Policy:** $a \sim \pi(a|s)$. Outputs a probability distribution over actions.

The fundamental objective of RL is to find the **optimal policy** (π^*).

3.2 Return (G_t)

RL algorithms seek to maximize cumulative rewards. The return G_t from time t is the sum of discounted future rewards:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots = \sum_{t'=t}^{\infty} \gamma^{t'-t} r_{t'} \tag{1}$$

If $\gamma < 1$, it ensures that infinite sums converge and gives preference to immediate rewards.

3.3 Expected Return and the Learning Objective

A single **Rollout** (or episode) generates one sample of the return. Because policies and environments can be stochastic, evaluating a policy requires looking at the **Expected Return**:

$$J(\pi) = \mathbb{E}_\pi[G] \tag{2}$$

Learning involves using data to search for the best policy—the one that yields the highest expected return.

4 Value-Based Reinforcement Learning

Value-based RL approaches the problem by estimating the expected return of states, then deriving the policy from these estimates.

4.1 State Value and Q-Value

- **State Value Function $V^\pi(s)$:** The expected return starting from state s and following policy π:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s] \tag{3}$$

- **State-Action Value Function (Q-value) $Q^\pi(s, a)$:** The expected return starting from state s, taking action a, and thereafter following policy π:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a] \tag{4}$$

Note that $V^\pi(s) = \mathbb{E}_{\pi(s)}[Q^\pi(s, \pi(s))]$.

4.2 Optimality

There exists an optimal value function associated with the optimal policy:

$$V^*(s) = \max_\pi V^\pi(s) \quad \forall s \in S \tag{5}$$

If we know the optimal Q-values, the optimal policy simply takes the action that maximizes the Q-value in the current state:

$$V^*(s) = \max_a [Q^*(s, a)] \implies \pi^*(s) = \arg\max_a Q^*(s, a) \tag{6}$$

4.3 Estimating Q-values: Monte-Carlo vs Bootstrap

To learn the Q-value, we rely on data (rollouts). There are two primary estimators:

1. **Monte-Carlo Estimator:**

- Runs simulations all the way to the end of the episode to observe the actual return G_t.
- Estimate $Q^\pi(s, a) \approx G_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \cdots + \gamma^n r_n$.
- **Properties:** High variance (due to accumulated stochasticity over the whole trajectory), but unbiased.

2. **Bootstrap Estimator (Temporal Difference / TD Learning):**

- Updates estimates based on other learned estimates, taking only a single step (or n-steps) in the environment.
- Based on the Bellman Equation: $Q(s,a) \approx r_0 + \gamma \max_b Q(s',b)$.
- **Properties:** Lower variance (relies on only one random transition), but higher bias (relies on an imperfect, developing estimate of future states).

5 Deep Reinforcement Learning

5.1 Function Approximators (FA)

Tabular Q-learning (storing a Q-value for every possible state-action pair in a matrix) becomes impractical for environments with large or continuous state spaces (e.g., images). To solve this, we use a **Function Approximator**, typically a Neural Network, to output Q-values: $Q(s,a;\theta) \approx Q^*(s,a)$.

The act of learning shifts from filling a table to searching for the optimal parameters (weights) θ of the function approximator.

5.2 Deep Q-Networks (DQN)

DQN uses a deep convolutional neural network as a function approximator. The network takes the state (e.g., screen pixels) as input and outputs a Q-value for every possible action.

The loss function uses the **Temporal Difference (TD) error**. According to Bellman's optimality equation, the estimator is correct if the left-hand side equals the right-hand side:

$$\hat{Q}(s_t) = \mathbb{E}_P[r_t + \gamma \max_b \hat{Q}(s_{t+1}, b)] \tag{7}$$

The TD-error is:

$$TD_Error = \left(r_t + \gamma \max_b \hat{Q}_\theta(s_{t+1}, b)\right) - \hat{Q}_\theta(s_t, a_t) \tag{8}$$

We optimize the weights θ by minimizing the squared TD-error:

$$L(\theta) = \left(r_t + \gamma \max_b \hat{Q}_\theta(s_{t+1}, b) - \hat{Q}_\theta(s_t, a_t)\right)^2 \tag{9}$$

6 Policy-Based Reinforcement Learning

Instead of implicitly deriving a policy from a value function (e.g., $\arg\max_a Q(s,a)$), **Policy Gradient** methods learn the policy directly. A Neural Network directly outputs the probability distribution of actions: $\pi_\theta(s) \to P(a|s)$.

Advantages of Policy-Based Methods:

- Can seamlessly learn continuous actions (whereas Q-learning struggles with $\arg\max_a$ over continuous spaces).
- Can yield non-deterministic (stochastic) policies.

6.1 The Policy Gradient Objective

The objective is to maximize the expected start-state return:

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G \mid s_0] = V^\pi(s_0) \tag{10}$$

Or the average-reward objective (where $d^\pi(s)$ is the stationary distribution of the Markov chain):

$$J(\theta) = \sum_s d^\pi(s) \sum_a \pi_\theta(s,a) r(s,a) \tag{11}$$

6.2 The REINFORCE Trick

Computing the gradient $\nabla J(\theta)$ directly is challenging because the distribution of trajectories depends on θ. Calculating differences by slightly perturbing θ is computationally expensive. Using the log-derivative trick (REINFORCE), we can sample trajectories using the current policy:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s,a) Q^{\pi_\theta}(s,a)] \tag{12}$$

Intuition: If $Q(s,a)$ is high (good action), the gradient pushes the network to increase $\log \pi_\theta(s,a)$ (making the action more likely). If $Q(s,a)$ is low/negative, the probability of that action is decreased. This allows us to optimize non-differentiable environments via sampling.

6.3 Baselines and Advantage Function

The raw policy gradient has very high variance, which slows down training. We can subtract a **Baseline** $b(s)$ from the Q-value without changing the expected gradient, effectively reducing variance. A convenient choice for the baseline is the State Value $V(s)$.

This leads to the **Advantage Function**:

$$A(s,a) = Q(s,a) - V(s) \tag{13}$$

The advantage function tells us the *relative* value of an action compared to the average expected value in that state. The new policy gradient update is (often referred to as A2C - Advantage Actor-Critic):

$$\nabla J(\theta) = \mathbb{E}_{s,a}\left[A(s,a)\nabla_\theta \log \pi_\theta(a|s)\right] \tag{14}$$

6.4 Encouraging Exploration (Entropy Regularization)

Early in training, a policy might become too confident in a suboptimal action (e.g., $\pi(a|s) = 1$). To prevent this and encourage exploration, we add an **Entropy term** $H(\pi_\theta)$ to the objective to punish low entropy:

$$\nabla_\theta J(\theta) \to \nabla_\theta J(\theta) + \nabla_\theta H(\pi_\theta) \tag{15}$$

7 Advanced Policy Optimization

7.1 Trust Region Policy Optimization (TRPO)

Standard RL updates can be noisy and destructive. If the policy changes too drastically based on an inaccurate batch of data, learning collapses. TRPO restricts the update size by ensuring the new policy doesn't diverge much from the old policy, measured by Kullback-Leibler (KL) divergence:

$$KL(\pi_{new} \parallel \pi_{old}) \leq \epsilon \tag{16}$$

This is solved using Lagrange multipliers and complex optimization techniques.

7.2 Proximal Policy Optimization (PPO)

PPO is a simplification of TRPO that achieves similar stability without the complex KL constraints. It modifies the policy gradient objective by clipping the probability ratio to keep the update within a trusted region. Let $r_t(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)}$. The PPO clipped objective is:

$$L^{CLIP}(\theta) = \mathbb{E}\left[\min\left(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)\right] \tag{17}$$

The min function ensures the loss ignores updates that would push the policy too far outside the $[1-\epsilon, 1+\epsilon]$ threshold. PPO is currently one of the default techniques used in industry (e.g., by OpenAI).

8 Taxonomies of RL Algorithms

8.1 On-Policy vs. Off-Policy

- **Off-Policy (e.g., Q-Learning):** Can learn the optimal policy using data generated by *any* policy. It evaluates the optimal path regardless of the agent's current exploratory actions.
  - Benefits: Sample efficient, can reuse old experience (Experience Replay) or expert data.
- **On-Policy (e.g., Policy Gradients, PPO):** The expected values must be computed over data generated by the *current* policy.
  - Drawbacks: Sample inefficient (data is discarded after the network updates).

8.2 Model-Free vs. Model-Based

- **Model-Free RL:** Learns strictly from trial-and-error interactions without knowing the transition dynamics P or reward function R. Learns purely via correlation.
- **Model-Based RL:** The agent first builds a mental model of the environment (predicting future states and rewards). It then uses this model to plan alternative routes.
  - AlphaGo is highly successful partly because the "model" (the rules of Go) is perfectly known, deterministic, fully observable, and static.
  - If uncertainty in the model is treated correctly, model-based RL is highly sample efficient. However, measuring uncertainty in dynamic real-world models is very difficult.

9 Challenges and Considerations in Modern RL

Despite its successes, RL faces substantial challenges:

1. **Data Inefficiency:** RL often requires millions of frames/interactions (e.g., 300,000x increase in compute from AlexNet to AlphaGo Zero). Many problems can be solved faster with supervised learning.
2. **Randomness:** Learning is highly sensitive to random initialization, random exploration, and environment stochasticity.
3. **Credit Assignment Problem:** Actions can have delayed consequences. An action taken at $t = 0$ might be responsible for a reward at $t = 10$, but the agent must correctly attribute the reward back through the intermediate actions.
4. **Sparse Rewards:** Because model-free RL depends on the correlation of trajectories and rewards, an environment that rarely issues a reward makes learning nearly impossible.
   - Potential fixes: Curiosity/Intrinsic motivation, Curriculum learning, Hierarchical RL.
5. **Designing the Reward Signal (Reward Gaming):** Naive reward design often leads to cheating behavior.
   - Example: CoastRunners (OpenAI), where the agent found a loop of collecting power-ups to rack up an infinite score instead of finishing the race.
   - Note: LLMs like ChatGPT sounding confident even when wrong is also a form of reward gaming (maximizing human approval metrics over factual correctness).
6. **Optimization Problems:** Issues with catastrophic forgetting and the limitations of Stochastic Gradient Descent (SGD) in dynamic non-stationary learning landscapes.